

מפת ארכיטקטורה מקיפה — SportOut

מטרת המסמך: סקירה מלאה של כל הקבצים והתיקיות ב-Repository עבור המייסד, כולל מטרה, לוגיקה, וחיבוריות לשאר המערכת.

מבנה כללי של הפרויקט

SportOut/	
app/	← Backend: FastAPI + SQLAlchemy
main.py	← תרשיל הסינכרונציה
core/	← Enums, DB, היצורגפונק
models/	← ORM ילדומ (ב-DB תואלבת)
schemas/	← Pydantic לש היצדילו (תבוסת/תושקב)
api/	← FastAPI (Routers) יביתו
sports/	← טרופסל תויפיצפס הקיטסיטסס תומכס
engines/	← גניקיימ'צאמו גוריד יעונמ
adapters/	← מוליצ יקפס (Pixellot, Manual)
workers/	← עקר תומישמ Celery
alembic/	← DB תויצרנימ
frontend/	← שמתשמ קשממ (HTML + Tailwind + Leaflet)
Dockerfile	← רנייטנוק תרדגה
docker-compose.yml	← דחי DB + תרש תצרה
requirements.txt	← תויולת Python
test_pulse.py	← תוריהמ יטינס תוקידב
PRODUCT_SPEC.md	← אלמ רצומ טרפמ
ROADMAP.md	← חותיפל מיכרד תפמ

1. נקודת הכניסה — app/main.py

מטרה: יצירת אפליקציית FastAPI, רישום כל ה-Routers, ו-Middleware.

מה הוא מגדיר:

- מחבר את 4 ה-Routers: /api/v1/players, /api/v1/matches, /api/v1/facilities, /api/v1/media
- מגדיר CORS (גישה פתוחה לכל מקור — לסגור בפרודקשן)
- Endpoint של GET /health → מחזיר {"status": "ok", "version": "1.0.0"}
- ניהול מחזור חיים של מנוע הדירוג בהפעלה/כיבוי: lifespan()

חיבוריות: הדלת הראשית של כל המערכת. כל בקשת HTTP נכנסת דרכו.

2. ליבת המערכת — app/core

ניהול קונפיגורציה — config.py

מטרה: קריאת כל הגדרות המערכת מקובץ env. עם ערכי ברירת מחדל.

משתנים מרכזיים:

תפקיד	משתנה
חיבור ל-PostgreSQL	DATABASE_URL
...גוריתם הדירוג הפעיל (ברירת מחדל: elo_v1)	RATING_ENGINE
...הצילום הפעיל (ברירת מחדל: manual_upload)	FILMING_PROVIDER
מפתח חתימת JWT	SECRET_KEY

S3_BUCKET / AWS_*	אחסון ענן לוידיאו
CELERY_BROKER_URL	לתור משימות Redis

חיבוריות: נטען על ידי כל מודול אחר במערכת.

database.py — DB-חיבור ל

מטרה: הגדרת מנוע SQLAlchemy אסינכרוני ו-Session Factory.

מה הוא עושה:

- יוצר Pool של חיבורים (10 בסיסיים, 20 מקסימום)
- אוטומטי commit/rollback עם Session שמספק dependency injection: `get_db()`
- יורש ממנה ORM המחלקה הבסיסית שכל מודל: `Base`

חיבוריות: כל endpoint ב-API משתמש ב-`get_db()` (Depends).

enums.py — Enum הגדרות

מטרה: כל הקבועים הסמנטיים של הדומיין במקום אחד.

מרכזיים Enums:

- Sport: SOCCER, BASKETBALL
- MatchStatus: OPEN → CONFIRMED → IN_PROGRESS → COMPLETED → CANCELLED
- MatchFormat: PICKUP, SCRIMMAGE, LEAGUE_GAME, TRAINING (משפיע על משקל דירוג)
 - UserRole: PLAYER, FACILITY_MANAGER, COACH, ADMIN
 - FilmingProvider: PIXELLOT, MANUAL_UPLOAD, VEO
- MediaProcessingStatus: PENDING → PROCESSING → COMPLETE / FAILED
 - CourtSurface, CourtCondition: מצב המגרש הפיזי

security.py — JWT ואבטחה

מטרה: הצפנת סיסמאות ויצירת/אימות טוקנים.

פונקציות:

- `hash_password()` / `verify_password()`: bcrypt
- `create_access_token(user_id, role)`: JWT עם `sub`, `role`, `exp` תקף 60 דקות
- `decode_access_token()`: claims אימות וחילוץ

חיבוריות: נקרא על ידי `api/dependencies.py` לאימות כל בקשה מאובטחת.

3. מודלי הנתונים — /app/models

users טבלת — user.py

מטרה: ישות האימות הבסיסית.

שדות עיקריים: `id` (UUID), `email`, `hashed_password`, `role`, `is_active`, `is_verified` (ייחודי),

חיבוריות: כל שחקן, מנהל מתקן וכומר יורש מ-User.

players + player_follows טבלאות — player.py

מטרה: פרופיל שחקן מלא כולל גרף חברתי.

שדות עיקריים:

- display_name, username (יחודי), bio, avatar_url, banner_url
- location (PostGIS POINT) — מיקום גיאוגרפי
- follower_count, following_count — מעודכנים על ידי
- card_is_public — האם הפרופיל גלוי לכולם

timestamp עצמית — עוקב/נעקב עם M2M מפת player_follows.

חיבוריות: ה-Hub המרכזי. מחובר למשחקים, דירוגים, וידאו, ועמיתים.

sport_profiles טבלת — sport_profile.py

מטרה: פרופיל ספורט נפרד לכל שחקן לכל ספורט.

שדות עיקריים:

- current_rating (מיון: 1000, מוגן)
- primary_position, secondary_positions
- career_games, career_wins — סטטיסטיקות מצטברות
- is_public

אילוש ייחודי: (player_id, sport) — פרופיל אחד לספורט.

חיבוריות: מעודכן על ידי workers/ratings.py לאחר כל משחק; בסיס לדירוגי Leaderboard.

matches, match_rosters, match_result_confirmations, player_game_stats טבלאות — match.py

מטרה: הישות המרכזית — מייצגת אירוע משחק מלא.

שדות עיקריים — matches:

- sport, status, format, scheduled_at
- facility_id, court_id — מיקום פיזי
- min_rating, max_rating — סינון רמת מיומנות
- home_score, away_score, mvp_player_id — תוצאה
- result_confirmed_at — מתי הושגה קוורום אישורים

match_rosters: team שחקן ↔ משחק עם: confirmed, attended

match_result_confirmations: דירוג כשמגיע לקוורום Worker מי מהשחקנים אישר את התוצאה — מפעיל:

player_game_stats: (sports/registry). JSONB-סטטיסטיקות ספציפיות לספורט ב:

חיבוריות: מחבר מתקנים, שחקנים, מדיה, ודירוגים.

facilities, courts, availability_slots טבלאות — facility.py

מטרה: ניהול מתקנים ספורטיביים, מגרשים, וזמינות.

שדות עיקריים — facilities:

- location (PostGIS POINT, GiST Index) — חיפוש גיאוגרפי מהיר
- sports_supported (ARRAY) — ספורטים זמינים
- opens_at, closes_at, operating_days — שעות פעילות
- amenities (JSONB) — 'חניה, מקלחות, קפה וכו' —

courts: current_occupancy, lighting_on, condition — מגרש בודד עם סטטוס חי:

חלון זמן להזמנה, מקושר למשחק שהזמין: availability_slots.

חיבוריות: מקושר למשחקים ולחיפוש גיאוגרפי בממשק המפה.

media_sessions, highlights — media.py

מטרה: ניהול הקלטות וידאו וקטעי הייליט.

media_sessions: session ה כתובת, ספק הצילום, כתובת ה session — S3 אחד למשחק — ספק הצילום, כתובת ה session.

highlights: url, duration_seconds, event_tags (AI labels), view_count — קטע ספציפי לשחקן.

חיבוריות: הפלט של workers/video.py; ה-video_metrics משמש כקלט למנוע הדירוג.

rating_events — rating.py

מטרה: לוג בלתי-הפיך של כל שינויי הדירוג (Event Sourcing).

שדות עיקריים:

- rating_before, rating_after, delta

- של אלגוריתם Replay-לביקורת ו — algorithm_version

- source_type — מקור: MATCH_RESULT, PEER_REVIEW, VIDEO_ANALYSIS, COACH_ASSESSMENT

- breakdown (JSONB) — פירוט כל גורמי החישוב

עיצוב: append-only בלבד. הדירוג הנוכחי = השורה האחרונה. מאפשר replay מלא עם אלגוריתם חדש.

חיבוריות: נכתב על ידי workers/ratings.py; נקרא על ידי Profile history ו-Leaderboard.

peer_reviews — peer_review.py

מטרה: הערכת שחקנים זה את זה לאחר משחק.

שדות עיקריים:

- reviewer_id, reviewed_player_id, match_id

- ratings (JSONB): כדורגל — defending, attacking, teamwork, effort; כדורסל — defense, offense, communication, effort (5–1 כולם בסקלה)

- is_confirmed — רק ביקורות מאושרות משפיעות על הדירוג

אילוף ייחודי: (reviewer, reviewed, match) — ביקורת אחת לשחקן למשחק.

4. סכמות ולידציה — /app/schemas

מטרה: הגדרת החוזה המדויק של כל בקשה ותשובה ב-Pydantic v2 (API).

קובץ	ישויות מרכזיות
player.py	PlayerCreate, PlayerCardOut, PlayerProfi...
match.py	MatchCreate (ולידציה: min_rating ≤ max_r...
facility.py	FacilityCreate, FacilityOut, CourtOut, C...
media.py	HighlightOut, MediaSessionOut, ManualUpl...
rating.py	RatingEventOut, RatingHistoryOut, PeerRe...

חיבוריות: כל endpoint ב-API משתמש בסכמות אלו לאימות קלט ועיצוב פלט.

5. ספורטים — /app/sports

רישום ספורטים — registry.py

מטרה: נקודת מרכז אחת לכל סכמות הסטטיסטיקה לפי ספורט.

```
STAT_SCHEMAS = {
    Sport.SOCER: SoccerGameStats,
    Sport.BASKETBALL: BasketballGameStats,
}
הציר נמזב היצדילו # validate_game_stats(sport, raw_dict)
```

חיבוריות: נקרא על ידי endpoint הסטטיסטיקות ב-Matches Router ועל ידי Worker הדירוג.

סטטיסטיקת כדורגל — soccer.py

שדות: עמדה (GK/CB/FB/CM/WG/ST), דקות (0–120), גולים, בישולים, בעיטות, דיוק מסירות %, קילומטרז', כרטיסים.
ולידציה: בעיטות על השער $\geq$ סה"כ בעיטות.

סטטיסטיקת כדורסל — basketball.py

שדות: עמדה (PG/SG/SF/PF/C), דקות (0–60), נקודות, אסיסטים, ריבאונדים, גניבות, חסימות, טעויות, % זריקה, Plus/Minus.
שדה מחושב: $\text{rebounds_total} = \text{rebounds_offensive} + \text{rebounds_defensive}$

6. מנועים — /app/engines

ממשק מנוע דירוג — engines/rating/base.py

מטרה: הגדרת הסכם שכל מנוע דירוג חייב לעמוד בו (Abstract Base Class).

- קלט): שחקן, ספורט, דירוג נוכחי, ממוצע דירוג יריבים, תוצאת משחק, פורמט, סטטיסטיקות) RatingContext •
ציון עמיתים.
- מפורט breakdown, דירוג חדש, גרסת אלגוריתם, delta: RatingDelta (פלט) •

גרסה 1 Elo מנוע — engines/rating/elo_v1.py

מטרה: חישוב שינוי דירוג לפי מודל Elo מותאם לספורט.

אלגוריתם:

ציון צפוי: $1 / (1 + ((\text{יריב} - \text{שחקן}) / 400)^{10})$

ציון בפועל: $1.0 / (1.0 + \text{תיקו} / 0.5)$ הפסד=0.0

(משקל_פורמט $\times$ (בפועל - צפוי $\times$ K(32) משחק Delta

- LEAGUE_GAME=1.2, SCRIMMAGE=1.0, PICKUP=0.8, TRAINING=0.4

(עמיתים: $\pm 0.15 \times$ (ציון_עמיתים - 3.0 Delta

דירוג חדש = $\max(0, \text{סה"כ_delta})$

חיבוריות: נטען דינמית על ידי get_active_engine() לפי הגדרת RATING_ENGINE ב-.env.

ממשק מאצ'מייקינג — engines/matchmaking/base.py

מטרה: הגדרת ממשק לאלגוריתם המלצת משחקים (ממתין למימוש).

- MatchmakingQuery: ± 200) סבילות דירוג (10 ק"מ), רדיוס (10 ק"מ), סבילות דירוג
- MatchSuggestion: match_id, score (0–1), בונס היסטוריה, מרחק, מיומנות, מרחק, בונס היסטוריה

7. אדפטרים — app/adapters/filming

מטרה: הפשטת ספקי הצילום — המערכת לא תלויה בספק ספציפי.

manual_upload.py — (פעיל) העלאה ידנית

מה הוא עושה: כאשר אין מצלמה חכמה:

- start_session(): עם קידומת ID מחזיר "manual:"
- stop_session(): לא עושה כלום
- generate_highlights(): בלי עריכה highlight-עוטף את הוידאו המלא כ:
- extract_video_metrics(): CV מחזיר מילון ריק (ללא ניתוח)

pixellot.py — (ממתין) Pixellot אינטגרציית

מה הוא עושה: Stub בלבד — כל הפונקציות מחזירות NotImplementedError.
ממתין לאישור גישה ל-API (פתיחת שאלה: OQ-02).

8. Workers — app/workers/

workers/__init__.py — הגדרת Celery

מטרה: יצירת אפליקציית Celery עם Redis כ-Broker.

workers/ratings.py — דירוג Worker

מטרה: חישוב מחדש של דירוגים ברקע לאחר אישור תוצאה.

```
recalculate_match_ratings(match_id):
    טוען משחק + roster + סטטיסטיקות
    בונה RatingContext לכל שחקן
    קורא ל-engine.compute()
    כותב שורות RatingEvent חדשות (append-only)
    מעדכן SportProfile.current_rating
    (ניסיונות חוזרים: 3 פעמים, 30 שניות המתנה)
```

replay_ratings_for_algorithm(sport, new_version): Replay של כל ההיסטוריה עם אלגוריתם חדש

workers/video.py — וידאו Worker

מטרה: עיבוד וידאו מלא ברקע.

```
process_media_session(media_session_id):
    שולף שחקנים מה-roster
    קורא ל-provider.generate_highlights() (לכל שחקן)
    שומר שורות Highlight
    קורא ל-provider.extract_video_metrics()
    מעדכן MediaSession.video_metrics + processing_status
    מעביר לתור את recalculate_match_ratings אם יש מטריקות
    (ניסיונות חוזרים: 3 פעמים, 60 שניות המתנה)
```

dependencies.py — הגנת endpoints

פונקציות:

- `get_current_user()`: מאמת JWT Bearer Token, מחזיר TokenData
- `get_current_facility_manager()`: דורש role FACILITY_MANAGER/ADMIN
- `get_current_admin()`: דורש role ADMIN

players.py — נתיבי שחקנים

Endpoints:

תיאור	סטטוס	נתיב
רישום שחקן חדש	Stub	POST /players
8 שחקנים מדומים, ניתן לסנן לפי עיר	Mock	GET /players/leaderboard/{sport}
פרופיל שחקן מלא	Mock	GET /players/{player_id}
עדכון פרופיל	Stub	PATCH /players/{player_id}
היסטוריית דירוג	Stub	GET /players/{player_id}/rating/{sport}
קטעי הייליט	Stub	GET /players/{player_id}/highlights
מעקב/ביטול מעקב	Stub	POST/DELETE /players/{player_id}/follow
שליחת ביקורת עמית	Stub	POST /players/reviews

matches.py — נתיבי משחקים

Endpoints:

תיאור	סטטוס	נתיב
יצירת משחק	Stub	POST /matches
חיפוש (ספורט, מיקום, דירוג)	Stub	GET /matches
פרטי משחק	Stub	GET /matches/{id}
הצטרפות ל-roster	Stub	POST /matches/{id}/join
יציאה מ-roster	Stub	DELETE /matches/{id}/leave
דיווח תוצאה	Stub	POST /matches/{id}/result
אישור תוצאה → מפעיל Worker	Stub	POST /matches/{id}/result/confirm
הגשת סטטיסטיקות	Stub	POST /matches/{id}/stats

facilities.py — נתיבי מתקנים

Endpoints:

תיאור	סטטוס	נתיב
רישום מתקן	Stub	POST /facilities
...ם בתל אביב (Sportek, Dubnov, Padel Expo)	Mock	GET /facilities/nearby
פרטי מתקן	Stub	GET /facilities/{id}
עדכון מצב מגרש בזמן אמת	Stub	PATCH /facilities/courts/{id}/live

נתיבי מדיה — media.py

Endpoints:

תיאור	סטטוס	נתיב
העלאת וידאו ידנית, יוצר MediaSession	Stub	POST /media/upload
סטטוס עיבוד + highlights	Stub	GET /media/sessions/{id}
בודד highlight	Stub	GET /media/highlights/{id}
ציבורי/פרטי	Stub	PATCH /media/highlights/{id}/visibility

10. מיגרציות alembic — /DB

alembic/env.py

מטרה: הגדרת ריצת מיגרציה אסינכרונית — קורא DATABASE_URL מה-.env, מזהה את כל המודלים אוטומטית.

alembic/versions/3f8a1c9b27e4_initial_schema.py

מטרה: יצירת כל 15 הטבלאות + indexes + הפעלת PostGIS extension.

טבלאות:

users, players, player_follows, sport_profiles, facilities, courts, availability_slots, matches, match_rosters, match_result_confirmations, player_game_stats, media_sessions, highlights, rating_events, peer_reviews

11. Frontend — frontend/

index.html — אפליקציה ראשית

מטרה: ממשק מובייל-ראשוני עם מפה חיה, לוח דירוגים, ויצירת משחקים.

רכיבים:

- מפת Leaflet.js: גילוי מגרשים קרובים עם אנימציות ping
- לוח דירוגים: ציוני שחקנים דינמיים
- יצירת משחק: בחירת ספורט, מגרש, שעה Overlay
 - פרטי מגרש: תפוסה חיה, תנאים Overlay
 - הצטרפות, roster: פרטי משחק Overlay
 - Toast notifications: אנימציות כניסה/יציאה

עיצוב: רקע שחור (#000000), מבטאים ירוק ניאון (#39FF14), כרטיסים זכוכיתיים, פונט Inter.

profile.html — עמוד פרופיל שחקן

מטרה: פרופיל שחקן בסגנון bio, banner — Instagram, סטטיסטיקות, highlights, עוקבים.

12. קבצי קונפיגורציה

Dockerfile

- Base: Python 3.12-slim

- שרת: Uvicorn עם hot-reload
- פורט: 8000

docker-compose.yml

- db: postgres/postgis:16-3.4 — פורט 5432, volume קבוע
- app: Built מה-Dockerfile, port 8000, db-תלוי ב

requirements.txt

קטגוריה	ספריות
Web	FastAPI 0.110.3, Uvicorn
DB	SQLAlchemy 2.0, asyncpg, Alembic, GeoAlc...
ולידציה	Pydantic v2, email-validator
Auth	PyJWT, passlib[bcrypt]
Workers	Celery 5.4, Redis 5.1

13. בדיקות — test_pulse.py

מטרה: 6 בדיקות סניטי מהירות שמוכיחות שהמערכת מחוברת נכון.

- סריאליזציה כרטיס שחקן
- סכמת כדורגל תקינה
- סכמת כדורסל עם שדה מחושב
- דחיית נתונים לא תקינים ($\text{shots_on_target} > \text{shots_total}$)
- חישוב Elo (תרחיש ניצחון עם breakdown)
- סריאליזציה היסטורית דירוג
- הרצה: `python test_pulse.py`

14. מסמכי מוצר

PRODUCT_SPEC.md

- מפרט מלא (525 שורות) עם:
- חזון: "רקמת חיבור בין המגרש הפיזי לעולם הדיגיטלי"
- 5 עמודי תוכן: דירוג, מאצ'מייקינג, מדיה חכמה, אימון, מידע עירוני
- 8 שאלות פתוחות (08-QQ-01): אלגוריתם דירוג סופי, רישיון Pixellot, מונטיזציה, טווח ספורטים, שותפויות עירוניות, פרטיות וידאו לקטינים, אנונימיות ביקורת, עדכוני זמן אמת

ROADMAP.md

- 4 שלבים:
- שלב 1: Frontend ו-SSO UX, לוח דירוגים, 30 מגרשים אמיתיים
- שלב 2: Backend ואלגוריתמים (auth, דירוג 55-99, דיווח תוצאות, DB יחסי)
- שלב 3: DevOps והגדרת צוות
- שלב 4: עסקים וגו-טו-מרקט

15. סיכום סטטוס MVP

מוכן לחלוטין

- מודלי DB + מיגרציות (15 טבלאות)
- קונפיגורציה + אבטחת JWT
- סכמות Pydantic לכל ישות
- API Routers עם Mock data (leaderboard, facilities)
- סטטיסטיקות ספורטיביות (כדורגל + כדורסל)
- מנוע Elo v1 (מלא)
- אדפטר העלאה ידנית
- הגדרות Celery Workers
- Docker + docker-compose
- בדיקות pulse

ממתין למימוש (Service Layer)

- רישום שחקן + (/auth/token Login)
- כל לוגיקת CRUD של משחקים ומתקנים
- Worker (orchestration) חישוב דירוג
- Pipeline (orchestration) וידאו
- קוורום אישור תוצאה
- ניתוח CV ו-Pixellot

החלטות פתוחות

- OQ-01: אלגוריתם דירוג סופי
- OQ-02: Pixellot של API-גישה ל
- OQ-03: מודל מונטיזציה
- OQ-04: V1-טווח ספורטים ל